



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE — FALL 2021

HOMEWORK 3 - SOLUTION

Due: 11:59pm, Saturday, April 10
Submit to: engin604assignments@gmail.com
Maximum page limit: 5 pages

See the homework policies at <https://docenciaweb.fen.uchile.cl>



1. Import the files `stocks.pkl`, `sp500.pkl` and `rfree.pkl`.

- `stocks.pkl`: Facebook, Apple, Amazon, Netflix and Google indices from 2014:12 to 2020:12.
- `sp500.pkl`: Closing price of the S&P 500 index from 2014:12 to 2020:12.
- `rfree.pkl`: Risk-free rate from 2015:01 to 2020:12.

```
import numpy as np # loads numpy
import pandas as pd # loads pandas

# reads indices
stocks = pd.read_pickle('stocks.pkl')

# reads sp 500
sp500 = pd.read_pickle('sp500.pkl')

# reads risk-free rate
Rf = pd.read_pickle('rfree.pkl')
```

2. Repeat the exercise from the **Application: Capital Asset Pricing Model** guide (up to question 6) using logarithmic returns instead of arithmetic returns.

$$R_{i,t} = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right) \quad i = 1, \dots, 5 \quad t = 1, \dots, 73$$

Hint: NumPy has the `np.log()` function to compute the natural logarithm.

```
# the function that computes the logarithmic returns is defined
def returns(x):

    log_return = np.log(x[1:] / x[:-1])
    return(log_return)

# empty dictionary
dict_return = {}

# computes the logarithmic return and stores it in a dictionary
for ticker, close in stocks.items():
```

```

print('Computing return of ticker: ', ticker)
dict_return[ticker] = returns(close)

## Computing return of ticker: AAPL
## Computing return of ticker: AMZN
## Computing return of ticker: FB
## Computing return of ticker: GOOG
## Computing return of ticker: NFLX

# market return
Rm = returns(sp500)

# risk premium
Rm_Rf = Rm - Rf

# weights
w = np.array([0.25, 0.25, 0.20, 0.20, 0.10]).reshape((-1,1))

# monthly rebalanced portfolio
def portfolio(x, w, rf):

    ret_list = []

    for name, r_i in x.items():
        ret_list.append(r_i)

    ret_mat = np.concatenate(ret_list, axis=1)
    rp = ret_mat.dot(w) - rf

    return(rp)

Rp_Rf = portfolio(dict_return, w, Rf)

# ordinary least square (OLS)
def ols(x,y):

    m,n = x.shape

    x_with_ones = np.hstack((np.ones((m,1)),x))

    x_t_x = x_with_ones.T.dot(x_with_ones)
    inv_x_t_x = np.linalg.inv(x_t_x)

    betas = inv_x_t_x.dot(x_with_ones.T.dot(y))

    return(betas)

# the function is applied
ols(Rm_Rf,Rp_Rf)

## array([[0.01500113],
##        [1.10008131]])

```

3. Show that the CAPM $\hat{\beta}$ is equal to:

$$\hat{\beta} = \sum_{i=1}^{n=5} w_i * \hat{\beta}_i$$

Where w_i is the weight of asset i in the portfolio created in (2) and $\hat{\beta}_i$ the second estimated coefficient of the following simple linear regression:

$$R_{i,t} - R_f = \alpha_i + \beta_i(R_{m,t} - R_f) + \varepsilon_{i,t}$$

Where $R_{i,t}$ is the logarithmic return of asset i .

Hint: Remember that the guide contains the function to obtain $\hat{\beta}$ from its matrix form.

```
# empty list
Ri_list = []

# generates excess return
for name, r_i in dict_return.items():
    Ri_list.append(r_i - Rf)

# matrix of zeros
coef_mat = np.zeros((1,len(dict_return.items())))
coef_mat.fill(np.nan)

# regression and extracts the beta
for j, Ri in enumerate(Ri_list):
    coef = ols(Rm_Rf,Ri)
    coef_mat[0,j] = coef[1]

# weighted betas
coef_mat.dot(w)
```

```
## array([[1.10008131]])
```

4. Considering only the logarithmic return (without subtracting R_f) of the assets in (2):

a. Store the average return of each one in an *array*.

```
Er_list = [] # empty list

# appends the average of each return
for name, r_i in dict_return.items():
    Er_list.append(r_i.mean())

# matrix with expected returns
Er_mat = np.array(Er_list).reshape((1,len(Er_list)))
```

b. Compute the variance-covariance matrix.

```
Ri_without_Rf_list = [] # empty list

# appends the returns to the list
for name, r_i in dict_return.items():
    Ri_without_Rf_list.append(r_i)

# concatenates the list into a matrix
Ri_mat = np.concatenate(Ri_without_Rf_list, axis=1)

Cov_mat = np.cov(Ri_mat, rowvar=False) # computes the variance-covariance matrix
Cov_mat # the variance-covariance matrix is checked

## array([[0.00683221, 0.00314471, 0.00316614, 0.00249939, 0.00325511],
##        [0.00314471, 0.00674721, 0.00334641, 0.00314951, 0.00549686],
##        [0.00316614, 0.00334641, 0.00551225, 0.0028489 , 0.0032019 ],
##        [0.00249939, 0.00314951, 0.0028489 , 0.00386681, 0.00243238],
```

```
## [0.00325511, 0.00549686, 0.0032019 , 0.00243238, 0.01190223]])
```

5. The global minimum variance portfolio is the one with the lowest possible risk for a given level of return; in other words, there is no other portfolio for that given level of return that can have a lower risk. The optimization problem expressed in its matrix form is:

$$\min_{\mathbf{m}} \sigma_{p,m}^2 = \mathbf{m}'\Sigma\mathbf{m} \quad s.t \quad \mathbf{m}'\mathbf{1} = 1$$

Whose solution (also in matrix form) is:

$$\mathbf{m} = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}'\Sigma^{-1}\mathbf{1}}$$

Where $\mathbf{m}$ is the vector of optimal weights of the minimum variance portfolio, $\mathbf{1}$ a vector of ones whose dimension is $n \times 1$, with n being the total number of assets, and Σ^{-1} the inverse of the variance-covariance matrix ($n \times n$).

Using the matrix solution for $\mathbf{m}$, write a function whose *inputs* are: the vector of average returns and the variance-covariance matrix of the assets that would make up the minimum variance portfolio.

The *output* of the function must be the weights $\mathbf{m}$, the expected return and the standard deviation of the minimum variance portfolio.

- The expected return of the minimum variance portfolio is:

$$R_p = \mathbf{m}' \cdot \mu$$

Where μ is the vector of average returns of dimension $n \times 1$.

- The standard deviation of the minimum variance portfolio is:

$$\sigma_{p,m} = \sqrt{\mathbf{m}'\Sigma\mathbf{m}}$$

Test the function using the vector of returns created in (4.a) and the variance-covariance matrix created in (4.b). Explain your function line by line.

```
# the function to compute the minimum variance portfolio is defined
```

```
def min_var_port(mu, cov):
```

```
    m,n = mu.shape
```

```
    one_vec = np.ones((n,1))
```

```
    sigma_inv_mat = np.linalg.inv(cov)
```

```
    top_mat = sigma_inv_mat.dot(one_vec)
```

```
    bottom_scalar = one_vec.T.dot(sigma_inv_mat).dot(one_vec)
```

```
    wi = top_mat / bottom_scalar
```

```
    er = mu.dot(wi)
```

```
    sigma2 = (wi.T.dot(cov).dot(wi)) ** (1/2)
```

```
    return([wi, er, sigma2])
```

```
# the function is applied
```

```
min_var_port(Er_mat, Cov_mat)
```

```
## [array([[0.16243381],
```

```
## [0.00766798],
```

```
## [0.16612873],
```

```
## [0.59473828],
```

```
## [0.0690312 ]]), array([[0.01915554]]), array([[0.05806092]])])
```